Kubernetes Monitoring

What Monitoring Is

Monitoring is the practice of collecting, tracking, and alerting on predefined metrics and logs from your systems to know whether they're healthy, and to get notified when something breaks. In Kubernetes, this means watching the state of the cluster and everything running on it — nodes, pods, containers, the control plane, and your applications.

Monitoring answers questions you already know to ask: "Is CPU usage above 80%?", "Is this pod restarting repeatedly?", "Is the API server responding?"

What gets monitored in K8s

Cluster/infrastructure level: - Node health (CPU, memory, disk, network) - Control plane components (API server, scheduler, controller-manager, etcd) - Kubelet and container runtime health

Workload level: - Pod status (Running, CrashLoopBackOff, Pending, OOMKilled) - Deployment/ReplicaSet health (desired vs. available replicas) - Resource usage vs. requests/limits (CPU/memory) - Restart counts

Application level: - Request rate, error rate, latency (the "RED" metrics) - Custom business metrics

Common K8s monitoring stack

Component	Role
kube-state-metrics	Exposes K8s object state (pod status, deployment replicas, etc.) as metrics
metrics-server	Provides resource usage (CPU/memory) for `kubectl top` and HPA
Prometheus	Scrapes and stores time-series metrics
Alertmanager	Routes alerts (Slack, PagerDuty, email) based on Prometheus rules
Grafana	Visualizes metrics in dashboards

Example flow: Prometheus scrapes metrics from kube-state-metrics and application `/metrics` endpoints every 15–30s → stores them → alerting rules evaluate them → Alertmanager fires a Slack alert if, say, pod restart count > 5 in 10 minutes → Grafana dashboard shows the trend.

Observability (in Detail)

What It Is

Observability is a **property of a system**: how well you can understand its *internal state* just by examining its *external outputs* — without having to ship new code or guess. It's a superset of monitoring.

The key distinction:

- **Monitoring** tells you *whether* something is wrong (predefined dashboards/alerts for known failure modes).
- **Observability** lets you *ask new questions* about *why* something is wrong — including failure modes you never anticipated.

Monitoring is good at "known unknowns" (you set a threshold because you expected that failure). Observability is designed for "unknown unknowns" (a novel, unexpected failure pattern you couldn't have pre-built a dashboard for).

The Three Pillars of Observability

1. Metrics

Numeric, aggregated measurements over time (counters, gauges, histograms). - Example: `http_requests_total`, `container_memory_usage_bytes`, `pod_restart_count` - Strength: cheap to store, great for trends, dashboards, alerting thresholds - Weakness: aggregated — you lose per-request/per-event detail

2. Logs

Immutable, timestamped, discrete records of events — often unstructured or structured (JSON). - Example: application error logs, audit logs, container stdout/stderr - Strength: rich detail about a specific event - Weakness: high volume, expensive to store/search at scale, hard to correlate across services unless structured well (with trace/request IDs)

3. Traces

Show the full journey of a single request as it flows through multiple services/microservices, broken into "spans" (each span = one unit of work, e.g., one service call, one DB query). - Example: a user request hits the API gateway → auth service → order service → payment service → DB. A trace shows the time spent in each hop. - Strength: pinpoints *where* in a distributed chain the latency or error occurred - Weakness: requires instrumentation (e.g., OpenTelemetry SDKs) in your code

How These Three Work Together (Correlation)

The real power of observability isn't having metrics, logs, and traces in isolation — it's **correlating them**:

1. A metric alert fires: error rate on `checkout-service` spiked to 15%.
2. You pivot to **traces** filtered to that service/time window to see which downstream call is failing and how long it took.
3. You pivot to **logs** for that specific trace ID to see the exact error message/stack trace.

This "pivot from symptom to root cause across all three signals" is what distinguishes real observability from just having three separate tools.

Observability Tooling in Kubernetes

Signal	Common Tools
Metrics	Prometheus, Thanos/Mimir (long-term storage), Grafana
Logs	Fluentd/Fluent Bit → Loki or Elasticsearch (EFK/ELK stack)
Traces	Jaeger, Tempo, OpenTelemetry Collector
All-in-one	Grafana stack (Prometheus + Loki + Tempo + Grafana), or commercial: Datadog, New Relic, Dynatrace, Honeycomb

OpenTelemetry (OTel) deserves a special mention — it's becoming the vendor-neutral standard for instrumenting code to emit metrics, logs, and traces consistently, so you're not locked into one backend.

Key Practical Difference — Summary Table

	Monitoring	Observability
Goal	Detect known failure modes	Investigate unknown/novel issues
Approach	Predefined dashboards & alerts	Ad-hoc, exploratory querying
Data	Mostly metrics	Metrics + logs + traces, correlated
Question answered	"Is it broken?"	"Why is it broken, and where exactly?"
K8s example	Alert: pod restarted 5x in 10 min	Trace shows restart caused by OOM triggered by a memory leak introduced in last deploy, visible in logs at the exact timestamp

In short: monitoring is a subset of practices (metrics + alerting) that tells you *something* is wrong; observability is the broader engineering discipline/system design that makes it *possible to explain* why, by giving you rich, correlatable telemetry (metrics, logs, traces) instead of just green/red signals.

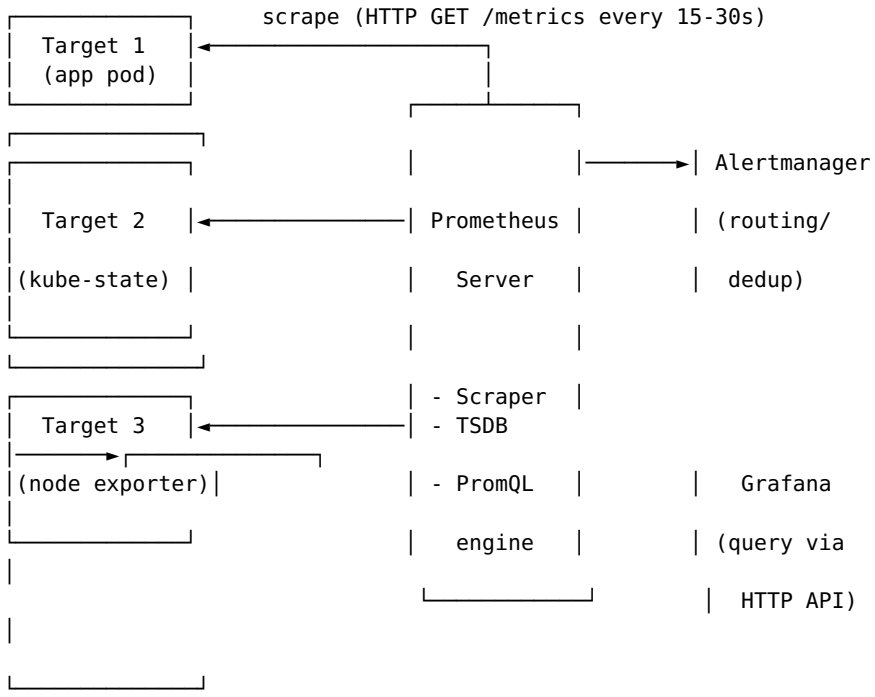
Prometheus — In Detail

What It Is

Prometheus is an open-source **metrics collection, storage, and alerting system**. It's the de facto standard for monitoring in Kubernetes (originally built at SoundCloud, now a CNCF graduated project — same maturity tier as Kubernetes itself).

The core model: Prometheus **pulls** (scrapes) metrics from targets over HTTP at regular intervals, rather than having applications push metrics to it.

Architecture & How It Works



1. Exporters / Instrumented Targets

Prometheus scrapes anything that exposes metrics on an HTTP endpoint (usually `/metrics`) in the Prometheus text format:

```
http_requests_total{method="GET",status="200"} 1027
container_memory_usage_bytes{pod="checkout-7f9d"} 52428800
```

- **Instrumented apps**: use a client library (Go, Java, Python, etc.) to expose their own metrics directly.
- **Exporters**: translate a third-party system's stats into Prometheus format when you can't instrument it directly (e.g., `node_exporter` for host-level CPU/disk/memory, `kube-state-metrics` for K8s object state, `mysqld_exporter` for MySQL).

2. Service Discovery

In K8s, Prometheus doesn't hardcode a list of IPs to scrape (pods are ephemeral, IPs change constantly). Instead it uses **Kubernetes service discovery** — it talks to the K8s API to auto-discover pods/services/endpoints that have the right annotations or match a `ServiceMonitor` (if using the Prometheus Operator), and keeps that target list live as pods scale up/down.

3. Scraping

On a configured interval (commonly 15s–60s), Prometheus pulls the current value of every metric from every discovered target. Each scrape is a snapshot — a full text response with all current metric values.

4. TSDB (Time Series Database)

Every metric is stored as a **time series**: a metric name + a set of key-value labels + a sequence of (timestamp, value) pairs.

```
http_requests_total{method="GET", status="200", pod="checkout-7f9d"}
```

This label model is what makes Prometheus powerful — you can slice/filter/aggregate by any label combination. Data is stored locally on disk (compressed, efficient for numeric time series), with a default retention window (commonly 15 days) — for longer retention people pair it with Thanos, Mimir, or Cortex.

5. PromQL (Prometheus Query Language)

The query language used to ask questions of the stored metrics:

```
# current error rate
rate(http_requests_total{status=~"5.."}[5m])

# average memory usage per pod over last hour
avg_over_time(container_memory_usage_bytes[1h])

# is any pod restarting a lot?
increase(kube_pod_container_status_restarts_total[15m]) > 5
```

PromQL supports rate calculations, aggregations (`sum`, `avg`, `max` by label), joins across metrics, and more — this is what powers both alerting rules and Grafana dashboards.

6. Alerting Rules → Alertmanager

You define rules in PromQL that, when true, fire an alert:

```yaml
- alert: PodCrashLooping
  expr: increase(kube_pod_container_status_restarts_total[10m]) > 3
  for: 5m
  labels:
    severity: critical
  annotations:
    summary: "Pod {{ $labels.pod }} is crash looping"
```

Prometheus evaluates this continuously and, when it fires, sends it to **Alertmanager**, a separate component that handles: - Deduplication (don’t spam 50 alerts for the same root cause) - Grouping (bundle related alerts into one notification) - Routing (send to Slack, PagerDuty, email based on severity/team labels) - Silencing/inhibition (mute known issues, suppress downstream alerts when a parent alert is already firing)

Push vs. Pull — Why It Matters

Prometheus’s pull model means: - Prometheus itself controls scrape timing/load (no target can overwhelm it) - Easy to tell if a target is down (a failed scrape = target down, itself a signal) - For short-lived jobs that finish before a scrape happens, there’s a **Pushgateway** as a workaround (jobs push their final metrics there, Prometheus scrapes the gateway instead)

Grafana — In Detail

What It Is

Grafana is an open-source **visualization and dashboarding layer**. It doesn't store metrics itself (by default) — it's a query frontend that connects to data sources (Prometheus being the most common, but also Loki for logs, Tempo for traces, Elasticsearch, InfluxDB, CloudWatch, SQL databases, etc.) and renders the results as graphs, tables, and dashboards.

Key point: Prometheus and Grafana are separate tools that pair together — Prometheus collects/stores/queries the raw data, Grafana visualizes it and layers on dashboards/alerting UI. Grafana itself has no scraping or storage engine of its own for metrics.

How It Works

1. Data Sources

You configure Grafana with a connection to Prometheus (its HTTP API endpoint, e.g. `http://prometheus:9090`). Grafana can connect to many data sources simultaneously — this is how you get **unified dashboards mixing metrics (Prometheus) + logs (Loki) + traces (Tempo)** on one screen.

2. Dashboards & Panels

A dashboard is made of panels; each panel runs a query (in Prometheus's case, PromQL) against a data source and renders it as: - Time series graphs - Gauges / single stat panels - Heatmaps - Tables - Bar charts, pie charts, etc.

Example panel query:

```
sum(rate(http_requests_total{status=~"5.."}[5m])) by (service)
```

rendered as a line graph showing error rate per service over time.

3. Variables / Templating

Dashboards can have dropdown variables (e.g., `$namespace`, `$pod`) that dynamically filter the underlying query — so one dashboard template works across every namespace/cluster instead of building one per environment.

4. Grafana Alerting

Newer Grafana versions also have their own built-in alerting engine (separate from, and now often overlapping with, Prometheus/Alertmanager) — you can define alert rules directly on panels and route notifications through "contact points" (Slack, email, PagerDuty, webhook, etc.).

5. Pre-built Dashboards

The K8s ecosystem has widely-used community dashboards (e.g., "Kubernetes / Compute Resources / Namespace (Pods)", "Node Exporter Full") that you import by ID rather than building from scratch — these expect metrics from kube-state-metrics + node_exporter + cAdvisor, which is why that trio is the standard companion set to Prometheus in K8s.

How They Work Together — End to End Flow in K8s

1. **kube-state-metrics** and **node_exporter** run as pods/daemonsets, exposing K8s object state and host metrics respectively.
2. Your application pods expose their own `/metrics` endpoint (via a Prometheus client library).
3. **Prometheus** discovers all these targets via K8s service discovery and scrapes them every 15–30s, storing time series in its TSDB.
4. **Alerting rules** in Prometheus continuously evaluate PromQL expressions; when true, they fire to **Alertmanager**, which routes a Slack/PagerDuty notification.
5. **Grafana** connects to Prometheus as a data source. An engineer opens a Grafana dashboard, sees a spike in a graph (e.g., pod restarts), and drills down using PromQL-backed panels and template variables (filter by namespace/pod) to isolate the problem — often pivoting from there into Loki (logs) or Tempo (traces) if those are also wired in as Grafana data sources.

Common Deployment Pattern

In real clusters, this is usually installed together via the **kube-prometheus-stack** Helm chart, which bundles: - Prometheus Operator (manages Prometheus/Alertmanager as CRDs — `ServiceMonitor`, `PodMonitor`, `PrometheusRule`) - Prometheus + Alertmanager - Grafana (pre-wired to Prometheus, with default K8s dashboards) - node_exporter + kube-state-metrics

This is why “Prometheus + Grafana” is treated almost as one phrase in K8s conversations — but architecturally they remain: **Prometheus = collect, store, query, alert on metrics** and **Grafana = visualize and dashboard (whatever data source you point it at)**.

A Full Observability Stack: All Three Pillars Working Together

Since we’ve already covered Prometheus (metrics) and Grafana (visualization), let’s now build out the *complete* picture — a stack that covers **metrics + logs + traces** and shows how they connect during a real incident.

The Full Stack (Grafana’s “LGTM” Stack — a popular open-source combo)

Pillar	Tool	Role
Metrics	Prometheus	Scrapes & stores numeric time-series data
Logs	Loki	Stores & indexes log lines (cheaply, by label — not full-text)
Traces	Tempo (or Jaeger)	Stores distributed request traces
Instrumentation	OpenTelemetry	Vendor-neutral SDK/agent that generates metrics,

	(OTel)	logs, and traces from your app code
Visualization	Grafana	Single UI to query and correlate all of the above
Log/trace shipping	Fluent Bit / Promtail / OTel Collector	Agents that collect and forward logs/traces from pods to their backend
Alert routing	Alertmanager	Dedupes/routes alerts from Prometheus (and optionally Grafana's own alerting)

(This is one popular combo — an equally common alternative swaps Loki/Tempo for the ELK/EFK stack for logs and Jaeger for traces, or uses commercial platforms like Datadog/New Relic/Honeycomb/Dynatrace that bundle all three signals in one proprietary backend.)

What Each New Piece Does

Loki — Log Aggregation

Loki stores logs but, unlike Elasticsearch, it **doesn't index the full text** of every log line — it only indexes metadata **labels** (like `namespace`, `pod`, `app`), the same label model Prometheus uses. This makes it much cheaper to run at scale. You query it with **LogQL** (deliberately similar syntax to PromQL):

```
{namespace="checkout", pod=~"checkout-.*"} |= "error"
```

This says: "give me all log lines from pods matching this selector that contain the word 'error'." Because indexing is label-based rather than full-text, Loki is lightweight to run but relies on labels being well-structured to find things quickly.

Promtail (or increasingly, the OTel Collector) runs as a DaemonSet on every node, tails container log files (`/var/log/pods/...`), attaches K8s metadata as labels, and ships them to Loki.

Tempo / Jaeger — Distributed Tracing

These store **traces** — the request-level journey across services. A trace is made of **spans**:

```
Trace: user checkout request (450ms total)
├─ span: api-gateway (10ms)
├─ span: auth-service (30ms)
├─ span: order-service (200ms)
│   └─ span: db query (180ms)  ← the bottleneck
└─ span: payment-service (210ms)
```

Tempo is Grafana's own trace backend (works well with the LGTM stack, keyed by trace ID rather than a full search index — so it's cheap to run at scale, but you generally need a trace ID, often obtained via a metric or log link, to jump in). Jaeger is the older, more independent CNCF project with a full search UI. Both need your app to be instrumented so it emits spans.

OpenTelemetry (OTel) — The Instrumentation Layer

This is the glue that makes correlation possible. Instead of using three separate libraries (one for metrics, one for logs, one for traces), you instrument your app once with OTel SDKs, and it emits all three signal types with **shared context** — critically, a **trace ID** gets attached to relevant log lines and linked to metrics exemplars. That shared trace ID is what lets you jump from a metric spike → to the exact trace → to the exact log line, instead of manually correlating timestamps across three separate UIs.

The **OTel Collector** is a standalone agent/pipeline (deployed as a DaemonSet or sidecar) that receives telemetry from apps and can process, batch, and forward it to any backend (Prometheus, Loki, Tempo, or a commercial vendor) — meaning you can swap backends without re-instrumenting your code.

How It All Works Together — A Real Incident Walkthrough

1. Alert fires (Metrics → Alertmanager) Prometheus's alerting rule detects checkout-service error rate > 5% for 5 minutes → fires to Alertmanager → Slack ping.

2. Engineer opens Grafana, looks at the metrics panel A Prometheus-backed dashboard shows the error rate spike started at 14:32. Grafana's panel has **exemplars** — dots on the graph linking directly to a sample trace ID from that exact time window.

3. Click through to the trace (Metrics → Traces) Clicking the exemplar jumps to Tempo, showing the full trace for one of the failing requests: gateway → auth → order-service → **payment-service (span shows a 5xx error, 3200ms duration — way above normal)**.

4. Click through to logs for that exact span (Traces → Logs) Grafana's trace view has a "logs for this span" link — because OTel attached the trace ID to log output — jumping straight to Loki, filtered to `trace_id="abc123"`, showing the exact stack trace / error message from payment-service at that moment: e.g., a downstream timeout connecting to a third-party payment gateway.

5. Root cause found — in under a minute, without grepping raw logs across five services.

Why This Design Matters

The reason this "three pillars + one glue layer + one UI" pattern is now standard is:

- **Metrics** tell you fast that *something* is wrong and roughly where (cheap, aggregated, good for alerting).
- **Traces** tell you *which specific hop* in a distributed call chain caused it (essential once you have more than a couple of services).
- **Logs** give you the *exact detail* — the stack trace, the error message, the payload — once you know where to look.
- **Correlation via shared IDs/labels** (trace ID, pod name, namespace) is what turns three separate tools into one coherent debugging workflow — this is really the difference between "we have three dashboards" and "we have observability."